Lesson 50: Q-Learning and the Unified View of TD Control

AI/ML Mathematics

Contents

1 Motivation and Overview **2**

2 References **3**

3 Q-Learning: Off-Policy TD Control **3**

4 Convergence of Q-Learning **4**

5 Maximization Bias and Double Q-Learning **5**

6 Expected SARSA **6**

6.1 Variance Reduction 6

6.2 Convergence of Expected SARSA 7

7 Unified View of TD Control **8**

8 Convergence Comparison **9**

9 Exercises **10**

10 Summary **11**

11 Further Reading **11**

1 Motivation and Overview

In Lesson 49 we developed SARSA, an on-policy TD control method that evaluates and improves the policy it actually follows — including its exploratory actions. This on-policy nature is a strength when exploration is costly, but it also means SARSA can only learn from data generated by the current policy. In many practical scenarios, however, we want to learn the *optimal* policy from data generated by a different, more exploratory *behavior policy*. Consider an autonomous vehicle collecting driving data with a safe, conservative exploration policy: we would like to use this data to learn the optimal (and potentially aggressive) driving policy, without ever executing that aggressive policy during training.

Q-learning resolves this by decoupling the behavior policy (which generates data) from the target policy (which is being learned). Its update rule replaces the sampled next action A_{t+1} in the SARSA target with $\max_{a'} Q(S_{t+1}, a')$ — the value of the *best* action, regardless of which action was actually taken. This seemingly small change has profound consequences: Q-learning converges to Q^* no matter what behavior policy is used (as long as it explores sufficiently), making it the first and most fundamental off-policy reinforcement learning algorithm.

After establishing Q-learning, we develop Expected SARSA, which replaces the sampled next action with an expectation over the policy, and then present the unified view showing that SARSA, Q-learning, and Expected SARSA are all instances of the same TD control template with different bootstrap targets. The tools developed in this lesson will let us prove the convergence of Q-learning via stochastic approximation and rigorously compare all three TD control methods.

Where Q-Learning and Its Variants Appear

- **Deep Q-Networks (DQN)** combine Q-learning with neural network function approximation, experience replay, and target networks — the breakthrough that launched deep RL.
- **Double Q-learning** addresses the maximization bias inherent in the max operator, leading to Double DQN.
- **Expected SARSA** generalizes both SARSA and Q-learning and forms the basis of the Rainbow DQN improvements.
- **Off-policy learning** is essential for experience replay (reusing past transitions) and learning from demonstrations, both foundational techniques in modern deep RL.

Prerequisites. Action-value Bellman operators $\mathcal{T}_Q^\pi$ and $\mathcal{T}_Q^*$ and their γ-contraction properties (Lesson 49, Lemma 3.3). SARSA update rule and its convergence under GLIE (Lesson 49, Definition 5.1 and Theorem 6.1). ε-greedy policies (Lesson 49, Definition 4.1). Stochastic approximation: the Robbins–Monro theorem with martingale noise and contraction stability condition (Lesson 31, Theorem 3.7). TD(0) prediction convergence machinery (Lesson 47).

Learning objectives.

- *State* the Q-learning update rule and explain the off-policy nature of the max operator in the bootstrap target.
- *Prove* the convergence of Q-learning to Q^* via the stochastic approximation framework.
- *Define* Expected SARSA and *prove* that it has lower variance than SARSA.
- *Construct* the unified TD control framework showing SARSA, Q-learning, and Expected SARSA as special cases.

- *Analyze* the maximization bias of Q-learning and explain the Double Q-learning correction.
- *Compare* the convergence properties, variance, and behavioral characteristics of all three TD control algorithms.

2 References

- Sutton and Barto [2018, Chapter 6, Sections 6.5–6.7] — Q-learning, Expected SARSA, and the comparison of TD control methods.
- Bertsekas and Tsitsiklis [1996, Chapter 6] — Convergence analysis of Q-learning and related value iteration methods.
- Watkins and Dayan [1992] — The original paper proving Q-learning convergence under appropriate conditions.
- Tsitsiklis [1994] — Asynchronous stochastic approximation framework providing the most general convergence proof for Q-learning.
- Jaakkola et al. [1994] — Alternative convergence proof using the contraction mapping approach.
- Szepesvári [2010, Section 4] — Compact and rigorous treatment of TD control convergence.

3 Q-Learning: Off-Policy TD Control

Definition 3.1 (Q-Learning Update Rule)**.** Given the transition $(S_t, A_t, R_{t+1}, S_{t+1})$, the *Q-learning update* is

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha_t \big[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\big]. \tag{1}$$

The TD error for Q-learning is $\delta_t = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)$.

Remark 3.2 (Off-Policy Nature). The critical distinction from SARSA is in the bootstrap target. SARSA uses $Q(S_{t+1}, A_{t+1})$ where A_{t+1} is drawn from the *behavior policy*. Q-learning uses $\max_{a'} Q(S_{t+1}, a')$, which evaluates the *greedy policy* regardless of which action was actually taken. The behavior policy (which generates transitions) can be *any* policy with sufficient exploration — it need not be the greedy or ε-greedy policy. This decoupling of behavior and target policies is the hallmark of off-policy learning.

Remark 3.3 (Q-Learning as Stochastic Value Iteration). Q-learning can be viewed as a sample-based, asynchronous version of Q-value iteration $Q \leftarrow \mathcal{T}_Q^* Q$. Value iteration applies $\mathcal{T}_Q^*$ synchronously to all (s, a) pairs using exact expectations; Q-learning applies it asynchronously to individually sampled pairs using stochastic estimates.

Example 3.4 (Q-Learning Update)**.** Consider the same 3×1 grid from Lesson 49 (Example 5.3) with $\mathcal{S} = \{L, C, R\}$, actions {left, right}, reward $+1$ at R, $\gamma = 0.9$, and $Q_0 \equiv 0$. Suppose at time t: $S_t = C$, $A_t =$ right (exploratory), $R_{t+1} = 1$, $S_{t+1} = R$. With $\alpha = 0.1$:

$$\begin{aligned} Q_1(C, \text{right}) &= 0 + 0.1[1 + 0.9 \cdot \max(Q_0(R, \text{left}), Q_0(R, \text{right})) - 0] \\ &= 0.1[1 + 0.9 \cdot 0] = 0.1. \end{aligned}$$

This is the same as the SARSA update in this case because $Q_0 \equiv 0$. The difference emerges later, when $Q(R, \cdot) \neq 0$: Q-learning uses $\max_{a'} Q(R, a')$ while SARSA uses the randomly sampled $Q(R, A_{t+1})$.

We now prove the central convergence result of this lesson.

4 Convergence of Q-Learning

Assumption 4.1 (Q-Learning Convergence Conditions).

1. The MDP has finite state space $\mathcal{S}$ and finite action space $\mathcal{A}$.
2. Rewards are bounded: $|r(s,a,s')| \le R_{\max}$ for all (s,a,s').
3. Every state–action pair (s,a) is visited infinitely often.
4. Step sizes satisfy $\sum_n \alpha_n(s,a) = \infty$ and $\sum_n \alpha_n^2(s,a) < \infty$ for each $(s,a) \in \mathcal{S} \times \mathcal{A}$.

Convergence of Q-Learning (Watkins & Dayan 1992)

Theorem 4.2 (Convergence of Q-Learning). *Under Assumption 4.1, the Q-learning iterates converge to the optimal action-value function almost surely:*

$$Q_t(s,a) \xrightarrow{a.s.} Q^*(s,a) \quad \textit{for all } (s,a) \in \mathcal{S} \times \mathcal{A}. \tag{2}$$

Proof. We follow Watkins and Dayan [1992] and Tsitsiklis [1994], casting Q-learning as a stochastic approximation and verifying all conditions of the Robbins–Monro theorem (Lesson 31, Theorem 3.7).

Step 1: Express as stochastic approximation. At the k-th visit to state–action pair (s,a) (occurring at global time t_k), the Q-learning update reads:

$$Q_{k+1}(s,a) = Q_k(s,a) + \alpha_k(s,a)\big[(\mathcal{T}_Q^* Q_k)(s,a) - Q_k(s,a) + w_k(s,a)\big], \tag{3}$$

where the noise is

$$w_k(s,a) = R_{t_k+1} + \gamma \max_{a'} Q_k(S_{t_k+1}, a') - (\mathcal{T}_Q^* Q_k)(s,a). \tag{4}$$

The Bellman optimality operator on Q-values is $(\mathcal{T}_Q^* Q)(s,a) = \sum_{s'} \mathcal{P}(s' \mid s,a)[r(s,a,s') + \gamma \max_{a'} Q(s',a')]$ (Lesson 49, Definition 3.2).

Step 2: Verify the contraction property. By Lesson 49, Lemma 3.3, $\mathcal{T}_Q^*$ is a γ-contraction in $\|\cdot\|_\infty$ with unique fixed point Q^*. The driving function $h(Q)(s,a) = (\mathcal{T}_Q^* Q)(s,a) - Q(s,a)$ has the unique root $Q = Q^*$.

For stability, we verify the contraction condition for the expected update. For any $\alpha \in (0,1]$:

$$\begin{aligned}
\|Q + \alpha\, h(Q) - Q^*\|_\infty &= \|(1-\alpha)Q + \alpha\, \mathcal{T}_Q^* Q - Q^*\|_\infty \\
&= \|(1-\alpha)(Q - Q^*) + \alpha(\mathcal{T}_Q^* Q - \mathcal{T}_Q^* Q^*)\|_\infty \\
&\le (1-\alpha)\|Q - Q^*\|_\infty + \alpha\gamma\|Q - Q^*\|_\infty \\
&= (1 - \alpha(1-\gamma))\|Q - Q^*\|_\infty.
\end{aligned} \tag{5}$$

Since $\alpha(1-\gamma) > 0$, the expected update contracts the error, confirming that h drives iterates toward Q^*.

Step 3: Verify martingale noise. Let $\mathcal{F}_k$ denote the filtration containing all information up to the k-th visit to (s,a) (including Q_k and $S_{t_k} = s, A_{t_k} = a$). Then:

$$\begin{aligned}
\mathbb{E}[w_k(s,a) \mid \mathcal{F}_k] &= \mathbb{E}\big[R_{t_k+1} + \gamma \max_{a'} Q_k(S_{t_k+1}, a') \mid S_{t_k} = s, A_{t_k} = a, Q_k\big] - (\mathcal{T}_Q^* Q_k)(s,a) \\
&= \sum_{s'} \mathcal{P}(s' \mid s,a)\big[r(s,a,s') + \gamma \max_{a'} Q_k(s',a')\big] - (\mathcal{T}_Q^* Q_k)(s,a) \\
&= (\mathcal{T}_Q^* Q_k)(s,a) - (\mathcal{T}_Q^* Q_k)(s,a) = 0.
\end{aligned} \tag{6}$$

Thus $\{w_k(s,a)\}$ is a martingale difference sequence with respect to $\{\mathcal{F}_k\}$.

Step 4: Bound the noise variance. Since $|R_{t+1}| \leq R_{\max}$ and the Q-iterates remain bounded by $R_{\max}/(1-\gamma)$ (established by induction using the contraction (5)):

$$|w_k(s,a)| \leq R_{\max} + \frac{\gamma R_{\max}}{1-\gamma} + \frac{R_{\max}}{1-\gamma} = \frac{(2-\gamma)R_{\max}}{1-\gamma}. \tag{7}$$

Therefore $\mathbb{E}[w_k^2(s,a) \mid \mathcal{F}_k] \leq C$ for a constant $C = [(2-\gamma)R_{\max}/(1-\gamma)]^2$.

Step 5: Apply the Robbins–Monro theorem. All five conditions of the Robbins–Monro theorem are satisfied:

1. Step sizes: $\sum_k \alpha_k(s,a) = \infty$ and $\sum_k \alpha_k^2(s,a) < \infty$ by assumption.
2. Martingale noise: $\mathbb{E}[w_k \mid \mathcal{F}_k] = 0$ (Step 3).
3. Unique root: $h(Q) = 0$ if and only if $Q = Q^*$ (Step 2).
4. Stability: the contraction (5) ensures the expected update contracts toward Q^*.
5. Bounded noise: $\mathbb{E}[w_k^2 \mid \mathcal{F}_k] \leq C$ (Step 4).

Therefore $Q_k(s,a) \to Q^*(s,a)$ almost surely for all $(s,a) \in \mathcal{S} \times \mathcal{A}$. □

Remark 4.3 (No GLIE Needed). Unlike SARSA, Q-learning does not require the GLIE condition. The behavior policy need not converge to greedy — it only needs to visit every state–action pair infinitely often (Assumption 4.1, Condition 3). This is a weaker requirement than GLIE's Condition 2 ($\varepsilon_t \to 0$). In SARSA, the policy must converge to greedy so that the exploration perturbation vanishes (Lesson 49, proof of Theorem 6.1, Step 5); in Q-learning, the max operator directly targets Q^* without any perturbation.

Example 4.4 (Off-Policy Data Collection)**.** Suppose an agent explores with a uniformly random policy $\pi_b(a \mid s) = 1/|\mathcal{A}|$ for all (s,a). Q-learning applied to the resulting transitions still converges to Q^*, because Assumption 4.1 is satisfied: the uniform policy visits all state–action pairs infinitely often. SARSA, by contrast, would converge to Q^{π_b} (the value of the random policy), which is far from optimal.

From Q-Learning to DQN

Q-learning's convergence guarantee holds for the tabular case. When combined with neural network function approximation (DQN, Sutton and Barto [2018, Chapter 16]), two additional techniques are needed to maintain stability:

- **Experience replay**: stores transitions in a buffer and samples mini-batches for updates, breaking temporal correlation and enabling off-policy reuse.
- **Target networks**: uses a slowly updated copy of the Q-network for the bootstrap target $\max_{a'} Q_{\text{target}}(s',a')$, reducing the moving-target problem.

Both techniques exploit Q-learning's off-policy nature: experience replay reuses past transitions (generated by earlier policies), and the target network evaluates a frozen (hence different) policy.

5 Maximization Bias and Double Q-Learning

The max operator in Q-learning introduces a systematic positive bias when the Q-values are noisy estimates.

Proposition 5.1 (Maximization Bias)**.** *Let $X_1, \ldots, X_n$ be random variables with $\mathbb{E}[X_i] = q_i$. Then*

$$\mathbb{E}\left[\max_i X_i\right] \geq \max_i \mathbb{E}[X_i] = \max_i q_i. \tag{8}$$

Equality holds if and only if $\max_i X_i = X_{i^*}$ almost surely for the same index i^* (i.e., the maximizer is deterministic) or all X_i are equal almost surely.

Proof. For any fixed index j: $\max_i X_i \geq X_j$. Taking expectations: $\mathbb{E}[\max_i X_i] \geq \mathbb{E}[X_j] = q_j$. Since this holds for all j, $\mathbb{E}[\max_i X_i] \geq \max_j q_j$.

For the equality condition, define $Y = \max_i X_i - X_{i^*}$ where $i^* = \arg\max_i q_i$. Then $Y \geq 0$ and $\mathbb{E}[Y] = \mathbb{E}[\max_i X_i] - q_{i^*}$. If $\max_i X_i = X_{i^*}$ a.s., then $Y = 0$ a.s. and equality holds. Otherwise, $\mathbb{P}(Y > 0) > 0$, implying $\mathbb{E}[Y] > 0$ and strict inequality. □

Example 5.2 (Maximization Bias in a Simple MDP)**.** Consider a state s with two actions, where the true values are $Q^*(s,1) = Q^*(s,2) = 0$, but the current noisy estimates are $Q(s,1) = 0.3$ and $Q(s,2) = -0.2$. Then $\max_a Q(s,a) = 0.3$, which overestimates the true value $\max_a Q^*(s,a) = 0$. Q-learning uses this inflated estimate in its bootstrap target, potentially propagating and amplifying the overestimation.

Definition 5.3 (Double Q-Learning)**.** *Double Q-learning* maintains two independent Q-value estimates Q^A and Q^B. At each step, with equal probability, one estimate is used to select the action and the other to evaluate it:

$$Q^A(S_t, A_t) \leftarrow Q^A(S_t, A_t) + \alpha\big[R_{t+1} + \gamma\, Q^B\big(S_{t+1}, \arg\max_{a'} Q^A(S_{t+1}, a')\big) - Q^A(S_t, A_t)\big], \quad (9)$$

with a symmetric update for Q^B chosen with probability 1/2.

Proposition 5.4 (Double Q-Learning Removes Maximization Bias)**.** *In Double Q-learning, the expected bootstrap target is*

$$\mathbb{E}\big[Q^B\big(S_{t+1}, \arg\max_{a'} Q^A(S_{t+1}, a')\big) \mid S_{t+1}\big], \quad (10)$$

which is an unbiased estimate of the value of the action selected by Q^A, rather than an overestimate of $\max_{a'} Q^*(S_{t+1}, a')$.

Proof. Let $a^* = \arg\max_{a'} Q^A(S_{t+1}, a')$. Since Q^A and Q^B are updated on independent samples (by the random assignment mechanism), $Q^B(S_{t+1}, a^*)$ is an unbiased estimate of the value of action a^* (not of $\max_{a'} Q^*(s, a')$). Specifically:

$$\mathbb{E}[Q^B(S_{t+1}, a^*) \mid S_{t+1}, Q^A] = Q^*(S_{t+1}, a^*) + \text{estimation error of } Q^B, \quad (11)$$

where the estimation error has zero mean under appropriate conditions. The key point is that Q^B does not select a^* — it merely evaluates it — so there is no selection-evaluation coupling that causes maximization bias. □

Q-Learning Can Significantly Overestimate Values

In MDPs with many actions and noisy rewards, maximization bias causes Q-learning to systematically overestimate Q^*, sometimes by large margins. This can lead to poor policy performance: the agent believes certain actions are much better than they actually are and repeatedly selects them. Double Q-learning (or its deep RL variant, Double DQN) is the standard remedy.

6 Expected SARSA

Expected SARSA replaces the sampled next action in the SARSA target with an expectation over the policy, reducing variance while maintaining a flexible on/off-policy character.

Definition 6.1 (Expected SARSA Update Rule)**.** The *Expected SARSA update* is

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha_t \left[R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t)\right]. \quad (12)$$

Remark 6.2 (Expected SARSA Subsumes Q-Learning). When π is the greedy policy with respect to Q, we have $\sum_{a'} \pi(a' \mid s')Q(s', a') = \max_{a'} Q(s', a')$, and Expected SARSA reduces to Q-learning. When π is the behavior policy (e.g., ε-greedy), Expected SARSA is on-policy and evaluates the actual behavior.

6.1 Variance Reduction

Expected SARSA Has Lower Variance Than SARSA

Proposition 6.3 (Variance Reduction). *For any policy π and any action-value function Q:*

$$\mathrm{Var}\left(R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a') \;\middle|\; S_t, A_t \right) \leq \mathrm{Var}(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) \mid S_t, A_t)\,. \tag{13}$$

Proof. By the law of total variance, conditioning on S_{t+1}:

$$\begin{aligned} &\mathrm{Var}(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) \mid S_t, A_t) \\ &= \mathbb{E}[\mathrm{Var}(\gamma Q(S_{t+1}, A_{t+1}) \mid S_{t+1}) \mid S_t, A_t] + \mathrm{Var}(R_{t+1} + \gamma \mathbb{E}[Q(S_{t+1}, A_{t+1}) \mid S_{t+1}] \mid S_t, A_t)\,. \end{aligned} \tag{14}$$

Since $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$, the conditional expectation is $\mathbb{E}[Q(S_{t+1}, A_{t+1}) \mid S_{t+1}] = \sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a')$. Therefore:

$$= \underbrace{\gamma^2 \mathbb{E}\big[\mathrm{Var}_{A_{t+1} \sim \pi}(Q(S_{t+1}, A_{t+1}) \mid S_{t+1}) \mid S_t, A_t\big]}_{\geq 0} + \mathrm{Var}\left(R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a') \;\middle|\; S_t, A_t \right). \tag{15}$$

The first term is the expected variance of Q over the action choice A_{t+1}, which is non-negative. The second term is exactly the variance of the Expected SARSA target. Thus the SARSA variance exceeds the Expected SARSA variance by the non-negative first term. □

Example 6.4 (Variance Comparison). Consider a state s' with $|\mathcal{A}| = 4$ and Q-values $(10, 10, 0, 0)$ under an ε-greedy policy with $\varepsilon = 0.2$ (so $\pi(a_1 \mid s') = 0.85$, $\pi(a_i \mid s') = 0.05$ for $i = 2, 3, 4$).

SARSA samples $A_{t+1} \sim \pi$, yielding $Q(s', A_{t+1})$ that is either 10 (with probability 0.90) or 0 (with probability 0.10). The conditional variance of $Q(s', A_{t+1})$ is $0.9 \cdot 10^2 + 0.1 \cdot 0^2 - (0.9 \cdot 10)^2 = 90 - 81 = 9$.

Expected SARSA deterministically computes $\sum_{a'} \pi(a' \mid s')Q(s', a') = 0.85(10) + 0.05(10) + 0.05(0) + 0.05(0) = 9.0$. This contributes zero variance from the action selection.

6.2 Convergence of Expected SARSA

Theorem 6.5 (Convergence of Expected SARSA). *Under the same conditions as SARSA (Lesson 49, Assumption ?? with π_t satisfying GLIE), Expected SARSA converges to Q^* almost surely. If the policy π is fixed, Expected SARSA converges to Q^π.*

Proof. The Expected SARSA update targets the Bellman expectation operator $\mathcal{T}_Q^\pi$ (Lesson 49, Definition 3.1):

$$(\mathcal{T}_Q^\pi Q)(s, a) = \sum_{s'} \mathcal{P}(s' \mid s, a) \left[r(s, a, s') + \gamma \sum_{a'} \pi(a' \mid s') Q(s', a') \right]. \tag{16}$$

This is a γ-contraction with fixed point Q^π (Lesson 49, Lemma 3.3).

The update noise is $w_t = R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a') - (\mathcal{T}_Q^\pi Q)(S_t, A_t)$. This satisfies $\mathbb{E}[w_t \mid S_t, A_t, Q] = 0$ because the randomness comes only from the transition S_{t+1} (the action sum is deterministic given S_{t+1}), and the expectation over S_{t+1} yields $(\mathcal{T}_Q^\pi Q)(S_t, A_t)$ by definition.

For fixed π, the Robbins–Monro theorem gives $Q_t \to Q^\pi$ a.s. Under GLIE ($\pi_t \to \pi^*_{\text{greedy}}$), Expected SARSA targets converge to $\max_{a'} Q(S_{t+1}, a')$, and the same perturbed stochastic approximation argument as in SARSA's convergence proof (Lesson 49, Theorem 6.1) yields convergence to Q^*. □

Expected SARSA in Practice

Expected SARSA often outperforms both SARSA and Q-learning empirically:

- Compared to SARSA: lower variance from eliminating the action sampling noise (Proposition 6.3).
- Compared to Q-learning: no maximization bias when π is not the greedy policy.
- The additional computational cost is $O(|\mathcal{A}|)$ per update (summing over actions), which is negligible for small action spaces.

7 Unified View of TD Control

All three TD control algorithms share a common structure that differs only in the bootstrap target.

Unified TD Control Framework

Theorem 7.1 (Unified TD Control Update). *SARSA, Q-learning, and Expected SARSA can all be written as*

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha_t \big[R_{t+1} + \gamma \,\text{Target}(S_{t+1}) - Q(S_t, A_t)\big], \tag{17}$$

where the target function varies across algorithms:

Algorithm	Target(S_{t+1})	***On/Off-Policy***	***Converges to***
SARSA	$Q(S_{t+1}, A_{t+1})$, $A_{t+1} \sim \pi$	*On-policy*	Q^* *(GLIE)*
Q-Learning	$\max_{a'} Q(S_{t+1}, a')$	*Off-policy*	Q^*
Expected SARSA	$\sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a')$	*Either*	Q^π *or* Q^*

Proof. The unified form follows by inspection of Definitions 5.1 (Lesson 49), 3.1, and 6.1. The convergence column follows from Lesson 49, Theorem 6.1, Theorem 4.2, and Theorem 6.5 respectively. □

Proposition 7.2 (Relationships Between the Targets). *The three targets satisfy the following relationships:*

1. $\mathbb{E}_\pi[Q(S_{t+1}, A_{t+1}) \mid S_{t+1}] = \sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a')$. *That is, the Expected SARSA target is the conditional expectation of the SARSA target.*
2. *If π is the greedy policy with respect to Q:* $\sum_{a'} \pi(a' \mid s') Q(s', a') = \max_{a'} Q(s', a')$. *That is, Expected SARSA reduces to Q-learning.*
3. *If $\pi = \pi_b$ (the behavior policy): Expected SARSA is on-policy, evaluating π_b.*
4. *The SARSA target is an unbiased but noisy estimate of the Expected SARSA target:* $\text{Var}(Q(s', A_{t+1}) \mid s') \geq 0$ *with equality iff $Q(s', \cdot)$ is constant or π is deterministic.*

Proof. 1. Direct computation: $\mathbb{E}_\pi[Q(S_{t+1}, A_{t+1}) \mid S_{t+1} = s'] = \sum_{a'} \pi(a' \mid s')Q(s', a')$.

2. If $\pi(a \mid s') = \mathbf{1}\{a = \arg\max_{a'} Q(s', a')\}$, then $\sum_{a'} \pi(a' \mid s')Q(s', a') = Q(s', \arg\max_{a'} Q(s', a')) = \max_{a'} Q(s', a')$.

3. By definition, on-policy means the target policy equals the behavior policy. If $\pi = \pi_b$, Expected SARSA's target evaluates π_b.

4. From (1), $Q(s', A_{t+1})$ is a sample from a distribution with mean $\sum_{a'} \pi(a' \mid s')Q(s', a')$. Its variance is zero iff $Q(s', a)$ takes the same value for all a with $\pi(a \mid s') > 0$, or π concentrates on a single action. □

Example 7.3 (Unified View on a Two-Action MDP)**.** Consider $S_{t+1} = s'$ with $Q(s', a_1) = 5$, $Q(s', a_2) = 3$, and ε-greedy policy with $\varepsilon = 0.1$. Then $\pi(a_1 \mid s') = 0.95$, $\pi(a_2 \mid s') = 0.05$.

Algorithm	**Target**	**Value**
Q-learning	$\max(5, 3)$	5.00
Expected SARSA	$0.95(5) + 0.05(3)$	4.90
SARSA (if $A_{t+1} = a_1$)	$Q(s', a_1)$	5.00
SARSA (if $A_{t+1} = a_2$)	$Q(s', a_2)$	3.00
SARSA (expected)	$\mathbb{E}[Q(s', A_{t+1})]$	4.90

The Expected SARSA target (4.90) lies between Q-learning (5.00) and the average SARSA target, but with zero action-sampling variance. Q-learning's target (5.00) exceeds Expected SARSA's (4.90), reflecting the greedy maximization.

Remark 7.4 (The Continuum of TD Control). The three algorithms form a spectrum parameterized by the "degree of greediness" in the bootstrap target:

- **SARSA**: samples from the behavior policy (most stochastic, on-policy).
- **Expected SARSA**: averages over the behavior policy (deterministic given S_{t+1}, flexible on/off-policy).
- **Q-learning**: maximizes over actions (most greedy, off-policy).

Expected SARSA smoothly interpolates: as $\varepsilon \to 0$ in the ε-greedy policy, Expected SARSA approaches Q-learning; for any fixed $\varepsilon > 0$, it evaluates the exploratory policy.

8 Convergence Comparison

Proposition 8.1 (Summary of Convergence Properties)**.** *The convergence properties of the three TD control algorithms differ along three axes:*

	SARSA	***Q-Learning***	***Expected SARSA***
Converges to	Q^* *(GLIE)*	Q^*	Q^π *or* Q^*
Needs GLIE?	*Yes*	*No*	*Depends on* π
Bellman operator	$\mathcal{T}_Q^{\pi_t}$	$\mathcal{T}_Q^*$	$\mathcal{T}_Q^\pi$
Update variance	*High*	*Medium*	*Low*
Max. bias	*No*	*Yes*	*No (if* $\pi \neq$ *greedy)*

Proof. The convergence targets and GLIE requirements follow from Theorems 6.1 (Lesson 49), 4.2, and 6.5. The Bellman operators follow from the definitions. The variance ordering Var(Expected SARSA) $\leq$ Var(SARSA) is Proposition 6.3. Q-learning's variance is intermediate because it eliminates the action-selection variance of SARSA (by using max) but introduces variance from the max operator itself when the estimates are noisy. The maximization bias column

follows from Proposition 5.1: the max in Q-learning causes positive bias, while the expectation in Expected SARSA and the sampling in SARSA do not. □

Example 8.2 (Algorithm Comparison on a Cliff-Walking Task)**.** Consider a 4×12 gridworld with a cliff along the bottom edge (positions $(1, 1)$ through $(1, 10)$). The start is $(1, 0)$ and the goal is $(1, 11)$. Stepping onto the cliff gives -100 reward and returns the agent to start; all other steps give -1. With $\varepsilon = 0.1$ and $\gamma = 1$:

- **Q-learning** learns the optimal path along the cliff edge (shortest), but the agent falls off frequently during training due to ε-greedy exploration.
- **SARSA** learns a safer path one row above the cliff, accounting for the 10% chance of random steps. Online performance during training is better.
- **Expected SARSA** behaves similarly to SARSA but with less variance in the updates, often achieving the best online performance.

After $\varepsilon \to 0$, all three converge to the same optimal policy.

9 Exercises

Exercise 9.1 (Q-Learning TD Error as Martingale Difference)**.** $(*)$
Let $\delta_t = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)$ be the Q-learning TD error, and suppose $Q = Q^*$. Show that $\mathbb{E}[\delta_t \mid S_t, A_t] = 0$, i.e., the TD error under the optimal Q-function is a martingale difference sequence.

Exercise 9.2 (Expected SARSA on a Simple MDP)**.** $(*)$
Consider $\mathcal{S} = \{A, B\}$, $\mathcal{A} = \{1, 2\}$, transitions: both actions in A lead to B; action 1 in B returns to A, action 2 in B stays in B. Rewards: $r(A, 1) = 1$, $r(A, 2) = 0$, $r(B, 1) = 0$, $r(B, 2) = 2$. Let $\gamma = 0.9$, $Q_0 \equiv 0$, and π be the ε-greedy policy with $\varepsilon = 0.2$. Compute the first Expected SARSA update for each state–action pair visited in the trajectory $(A, 1, 1, B, 2, 2, B)$.

Exercise 9.3 (Maximization Bias Quantification)**.** $(**)$
Let $Q_1, \ldots, Q_n$ be independent estimates of the same true value q, each with $\mathbb{E}[Q_i] = q$ and $\mathrm{Var}(Q_i) = \sigma^2$.

(a) Show that $\mathbb{E}[\max_i Q_i] \geq q$ with equality iff $\sigma = 0$ or $n = 1$.
(b) For the special case $n = 2$ and $Q_i \sim \mathcal{N}(q, \sigma^2)$, show that $\mathbb{E}[\max(Q_1, Q_2)] = q + \sigma/\sqrt{\pi}$.
(c) Conclude that the maximization bias grows with σ and n.

Hint for (b): Use the identity $\mathbb{E}[\max(X, Y)] = \mathbb{E}[X] + \mathbb{E}[(Y - X)^+]$ and the fact that $Y - X \sim \mathcal{N}(0, 2\sigma^2)$.

Exercise 9.4 (Double Q-Learning Convergence)**.** $(**)$
Show that Double Q-learning (Definition 5.3) converges to Q^* under the same conditions as standard Q-learning (Assumption 4.1). Specifically:

(a) Write the Double Q-learning update for Q^A in the stochastic approximation form $Q^A_{k+1} = Q^A_k + \alpha_k[h(Q^A_k, Q^B_k) + w_k]$ and identify h and w_k.
(b) Show that $h(Q, Q) = (\mathcal{T}^*_Q Q)(s, a) - Q(s, a)$ when both estimates agree.
(c) Argue that Q^A and Q^B converge to the same limit Q^* by a coupled stochastic approximation argument.

Exercise 9.5 (Variance Ordering Proof Extension)**.** $(***)$
Extend the variance comparison of Proposition 6.3 to the n-step case. Define n-step SARSA and n-step Expected SARSA targets, and prove that the Expected SARSA variant has lower variance for any $n \geq 1$.

Hint: Apply the law of total variance iteratively, conditioning on $(S_{t+1}, \ldots, S_{t+n})$ and then on the actions.

Exercise 9.6 (Constructing a Counterexample for Off-Policy Divergence)**.** $(***)$
Consider a two-state MDP with states $\{s_1, s_2\}$ and two actions. Construct specific transition probabilities and rewards such that Q-learning with a uniform random behavior policy converges in fewer iterations than SARSA with $\varepsilon_t = 1/t$ (ε-greedy). Conversely, construct a different MDP where SARSA with $\varepsilon_t = 1/t$ achieves lower cumulative regret during learning than Q-learning (by exploiting the on-policy safety advantage).

Hint: For the second part, make exploratory actions in certain states very costly.

10 Summary

- **Q-learning** updates action values using the target $R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$, directly targeting the Bellman optimality operator $\mathcal{T}_Q^*$ regardless of the behavior policy. It is the canonical off-policy TD control algorithm.
- **Q-learning converges to** Q^* almost surely under finite MDP, bounded rewards, infinite visitation of all state–action pairs, and Robbins–Monro step sizes (Theorem 4.2). No GLIE condition is needed — the behavior policy can be any sufficiently exploratory policy.
- **Maximization bias**: the max operator in Q-learning causes systematic overestimation of Q^* when Q-estimates are noisy (Proposition 5.1). **Double Q-learning** corrects this by decoupling action selection from action evaluation using two independent estimates.
- **Expected SARSA** uses the target $\sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a')$, which is the conditional expectation of the SARSA target given S_{t+1}. It has *lower variance* than SARSA (Proposition 6.3) and *no maximization bias* (when π is not fully greedy).
- The **unified TD control framework** (Theorem 7.1) shows that SARSA, Q-learning, and Expected SARSA share the same update structure $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[\delta_t]$, differing only in the bootstrap target at S_{t+1}.
- Expected SARSA **interpolates** between SARSA and Q-learning: it reduces to Q-learning when the target policy is greedy, and to on-policy evaluation when the target policy equals the behavior policy.
- All three algorithms operate on the same Bellman operator structure ($\mathcal{T}_Q^\pi$ or $\mathcal{T}_Q^*$), which are γ-contractions, providing the stability condition for convergence via stochastic approximation.

11 Further Reading

- **Lessons 51–53** extend TD control to the function approximation setting, where the deadly triad of function approximation, bootstrapping, and off-policy learning can cause divergence. Q-learning is particularly susceptible because it is both bootstrapped and off-policy.
- Watkins and Dayan [1992] is the original Q-learning paper, introducing the algorithm and proving convergence for the tabular case.
- Tsitsiklis [1994] provides the most general convergence proof using the asynchronous stochastic approximation framework. The treatment handles non-uniform visitation patterns and connects to the ODE method for stochastic approximation [Borkar, 2008].
- Jaakkola et al. [1994] gives an alternative convergence proof based on a direct contraction argument, which is more accessible but slightly less general.

- Sutton and Barto [2018, Chapter 6] provides extensive empirical comparisons of SARSA, Q-learning, and Expected SARSA on benchmark tasks including the cliff-walking and windy gridworld problems.
- For Deep Q-Networks (DQN), Double DQN, Dueling DQN, and related extensions, see Lessons in Phase 04 (Deep RL). The tabular convergence theory developed here provides the foundation for understanding why these deep RL methods work and when they can fail.
- Szepesvári [2010, Section 4] provides a concise survey of finite-time convergence rates for TD control methods, complementing the almost-sure convergence results proved here.

References

Dimitri P. Bertsekas and John N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.

Vivek S. Borkar. *Stochastic Approximation: A Dynamical Systems Viewpoint*. Cambridge University Press, 2008.

Tommi Jaakkola, Michael I. Jordan, and Satinder P. Singh. On the convergence of stochastic iterative dynamic programming algorithms. *Neural Computation*, 6(6):1185–1201, 1994.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

Csaba Szepesvári. *Algorithms for Reinforcement Learning*. Morgan & Claypool, 2010.

John N. Tsitsiklis. Asynchronous stochastic approximation and Q-learning. *Machine Learning*, 16(3):185–202, 1994.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.